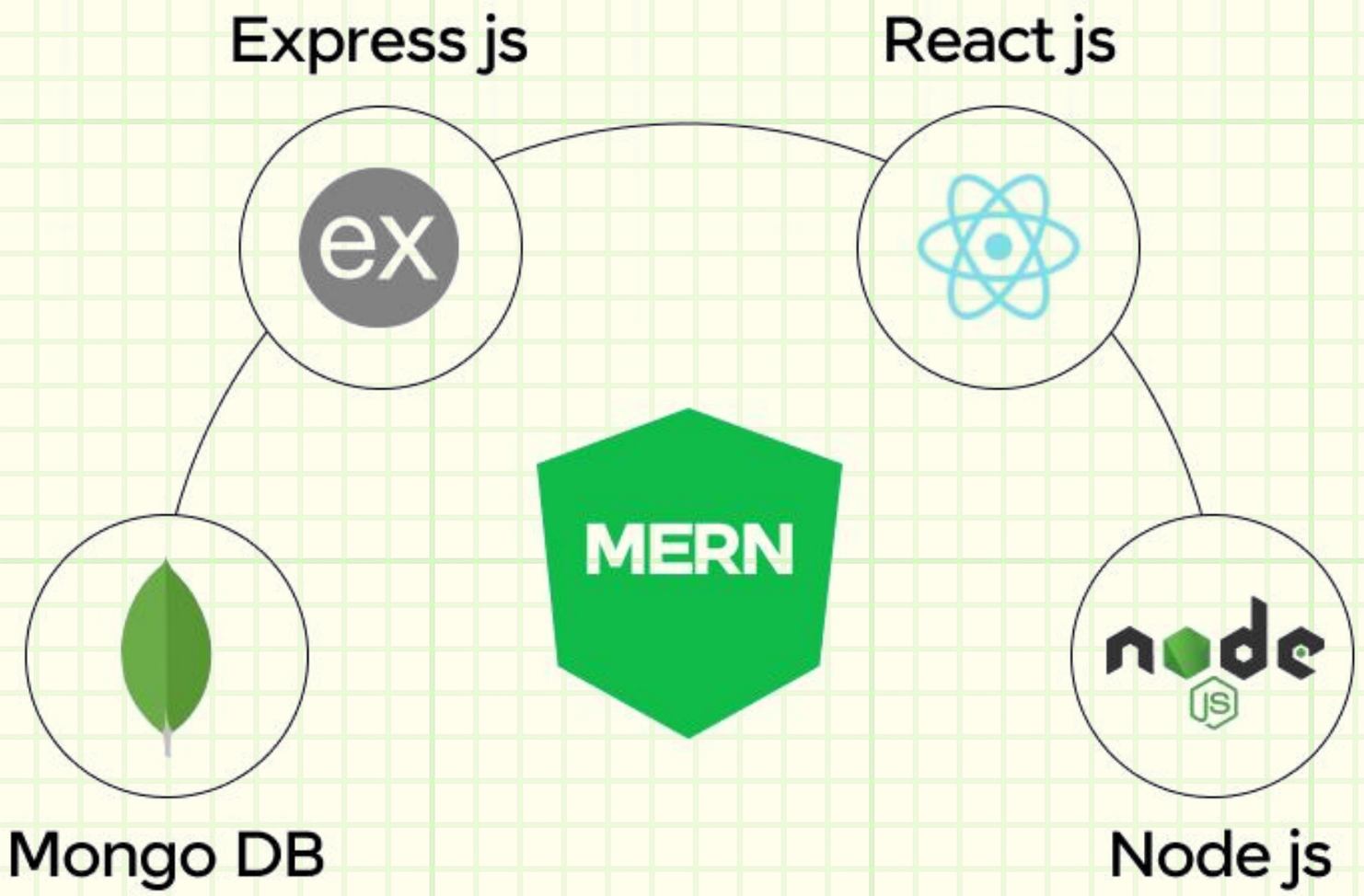
BOSSCODER ACADEMY

Zero To Hero

MERN Stack

ROADMAP



Complete Roadmap for MERN Stack Developer

Disclaimer

EVERYONE LEARNS UNIQUELY.

What matters is developing the problem solving ability
to solve new problems.

This Doc will help you with the same.

Introduction —

The MERN Stack is one of the most popular and powerful tech stacks used to build full-stack web applications. It stands for:

MONGO DB

NoSQL database

EXPRESS.JS

Backend web framework for Node.js

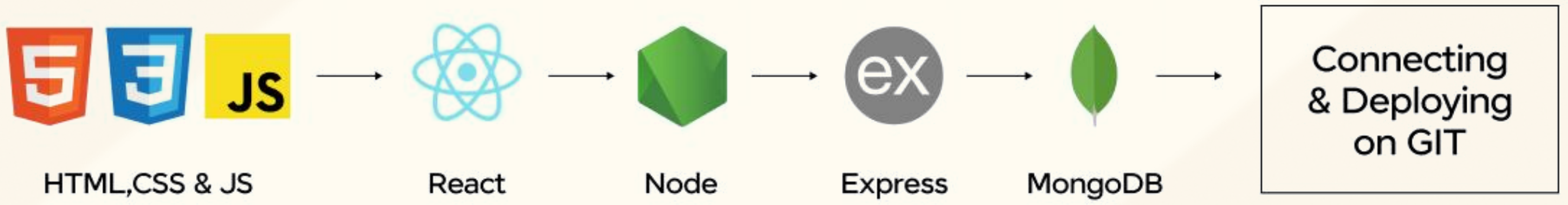
REACT.JS

Frontend library for building interactive UIs

NODE.JS

JavaScript runtime for executing server-side code

Note: Before diving into the MERN stack, it's essential to have a solid grasp of web development fundamentals.



Prerequisites —

01 HTML

KEY TOPICS:

1. HTML Tags & Elements

- Headings (<h1>-<h6>), Paragraphs (<p>) , Lists (<li>, <ul>,<ol>), Links (<a>)
- Semantic tags: <section>, <article>, <nav>, <footer>
- Global attributes: id, class

2. Forms & Input Elements

- <input>, <select>, <textarea>, <button>
- Form validation attributes (required, type, minlength etc.)

3. Media Elements

- <img>, <audio>, <video>
- attributes: controls, autoplay, loop, muted

4. Table and Layout Tags

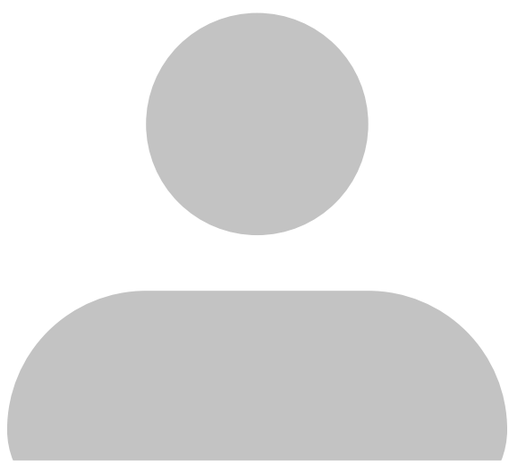
- <table>, <thead>, <tbody>, <tr>, <td>

PRACTICE:

Question 1 Create A Visiting Card →

A HTML project that demonstrates a basic layout with profile image and description section in the right with details **(Name, Profession, Email, Phone, Website).**

Reference :



John Doe

Profession: Web Developer

Email: Johndoe@example.com

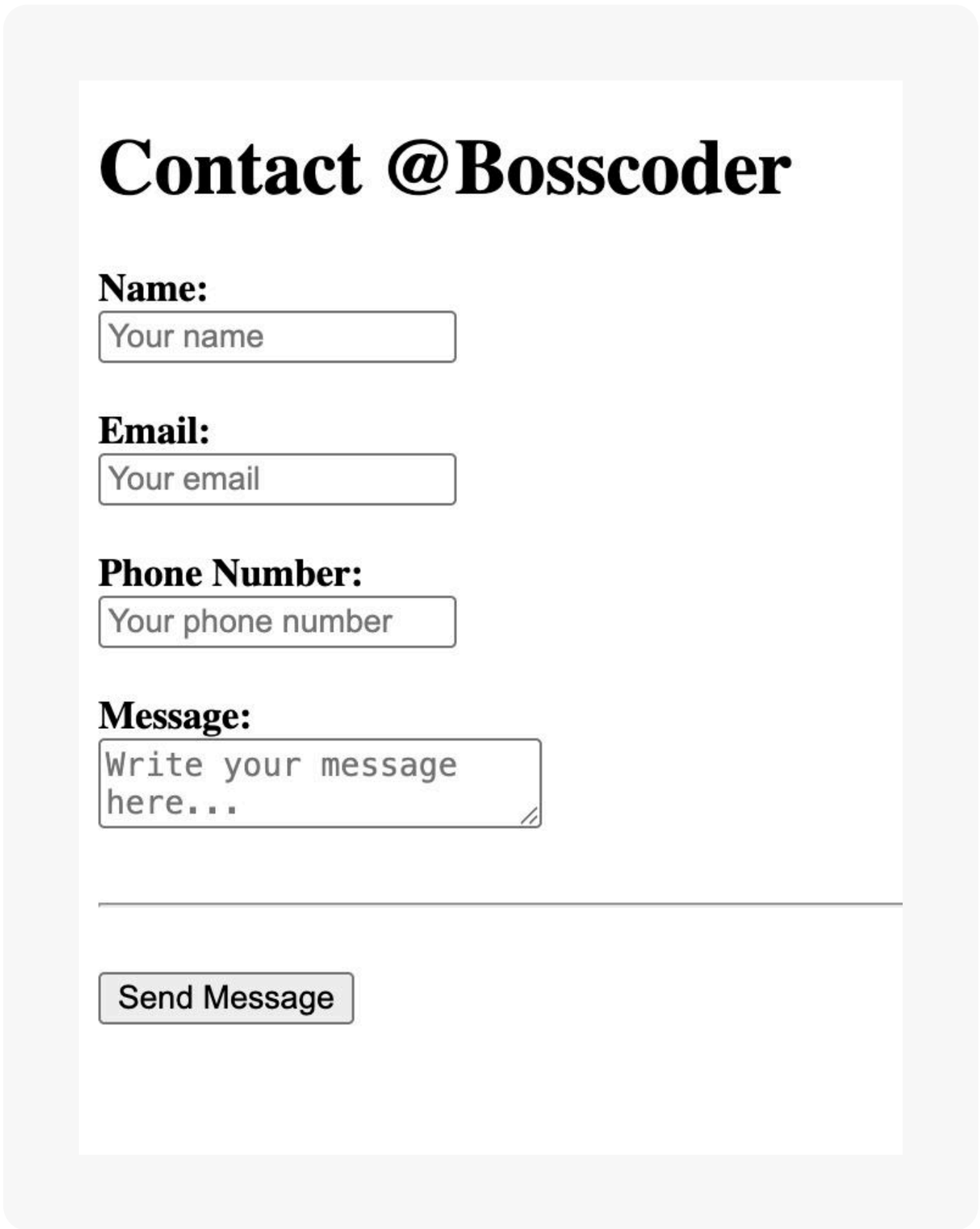
Phone: +123-456-7680

Website: www.example.com

Question 2 Create A Contact Form With Input Validation →

- A straightforward contact form with four fields—name, email, Phone and message—and a single send button.
- Implements HTML5 validation using `required, maxlength, pattern="[0-9]{10}",` and `title` attributes to ensure proper user input and provide helpful error messages.

Reference :


Contact @Bosscoder
Name:
Your name
Email:
Your email
Phone Number:
Your phone number
Message:
Write your message here...
Send Message

02 CSS

KEY TOPICS:

1. Selectors

- Basic Selectors: element, .class, #id
- Attribute Selectors: [type="text"], [href^="https"]
- Pseudo-classes: :hover, :focus, :nth-child()
- Pseudo-elements: ::before, ::after

2. Styling

- Typography: font-family, font-size, font-weight, line-height
- Colors: color, background-color, rbga(), gradients
- Borders and Shadows
- Transitions & Animations: transition, @keyframe, translation

3. Positioning & Layout

- position: static, relative, absolute, fixed, sticky
- display: block, inline, none, flex, grid
- Float, clear
- z-index

4. Flexbox & Grid

- parent: display: flex, justify-content, align-items, flex-direction
- child: flex-grow, flex-shrink, align-self

5. Responsive Design

- Media Queries: @media (max-width: 768px) {... }
- Units: %, em, vw, vh
- Responsive Images: max-width, height: auto

PRACTICE:

Question 1 Responsive Photo Gallery →

- Create a fully responsive photo gallery using only HTML and CSS, also add description below that photo.
- Use CSS Grid and Flexbox to organize images in a dynamic layout that automatically adjusts based on screen size.
- Display 3 columns on desktop and 2 columns on mobile devices

Reference :

03 JavaScript

KEY TOPICS:

1. Fundamentals

- Variables: let, const, var
- Data Types:
    i. Primitive: String, Number, Boolean, Null, Undefined
    ii. Complex: Object, Array, Function
- Operators: +, -, %, *, !, ==, !=, ===, typeof, instanceof

2. Functions & Scope

- Function declaration, parameters, return
- Arrow functions, Higher-order functions: Callbacks, Functions returning functions
- Scope: Global, Local, Block, Hoisting
- Closures, Lexical scope
- Array methods: .map(), .filter(), .reduce()

3. DOM Manipulation

- Selecting Elements: getElementById(), querySelector()
- Modifying DOM: innerHTML, createElement(), appendChild(), removeChild(), classList
- Events: addEventListener(), click, submit, Event object

4. Objects & Arrays

- Object Literals: Properties, Methods, this, Destructuring, Spread operator
- Arrays:
    i. Methods: .push(), .pop(), .slice(), .splice()
    ii. Iteration: .forEach(), .find(), .some(), .every()
- JSON: JSON.parse(), JSON.stringify()

5. Asynchronous JavaScript

- Timers: setTimeout(), setInterval()
- Promises: .then(), .catch(), .finally()
- async/await syntax
- Fetch API: fetch(), GET/POST requests, Handling JSON

6. Error Handling

- try/catch/finally
- Error types: ReferenceError, TypeError, etc.

7. Browser Storage

- localStorage, sessionStorage

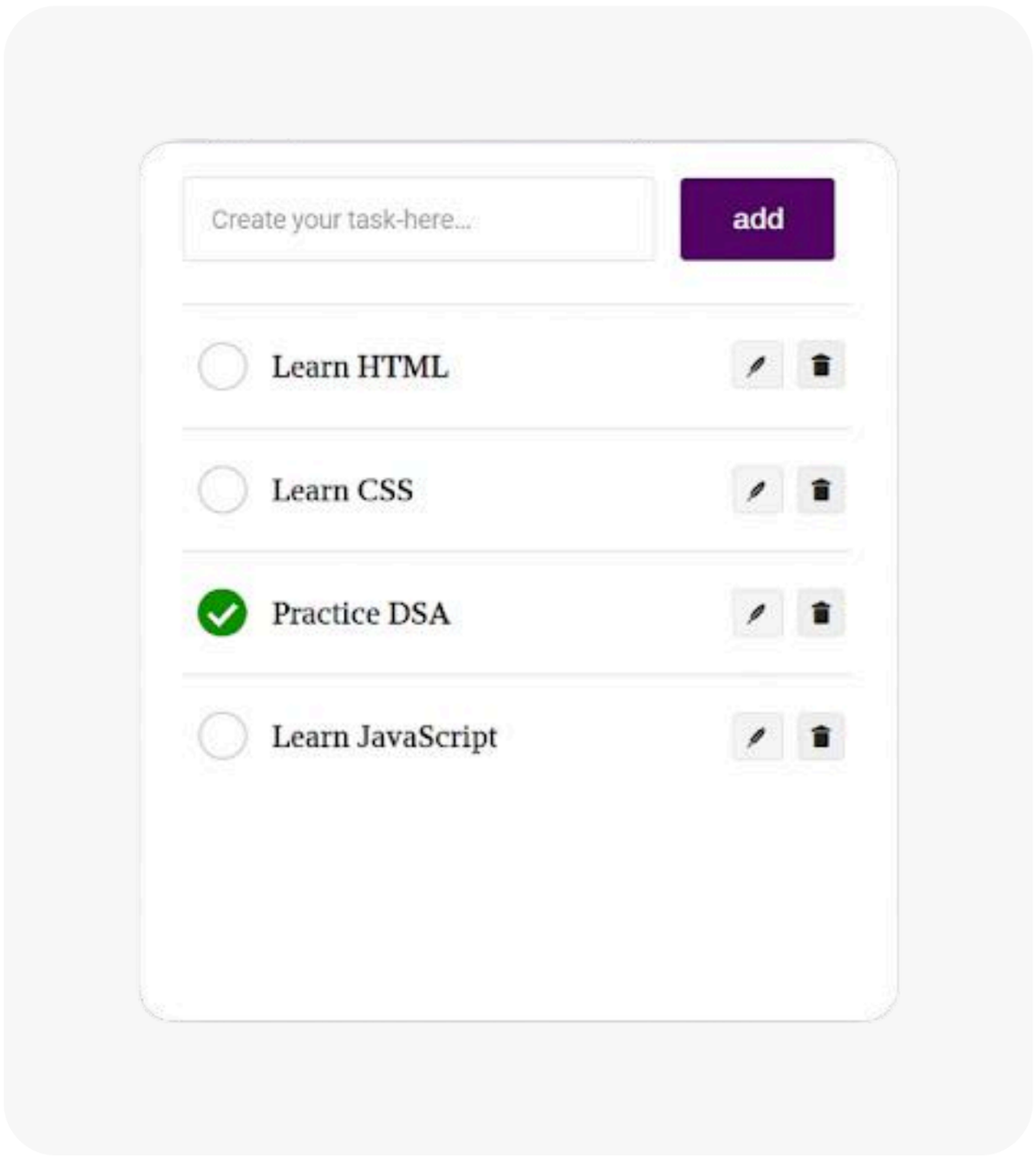
PRACTICE:

Question 1 ToDo List →

- Build a simple To-Do List App where users can Add, Edit tasks, Delete tasks Save and persist tasks using local storage (so tasks are not lost after page refresh)

- The UI has a text input and an Add button at the top. Below it, tasks appear in a list, each with Edit and Delete buttons. Clicking Edit makes the task text editable, and Delete removes it from the list.

Reference :

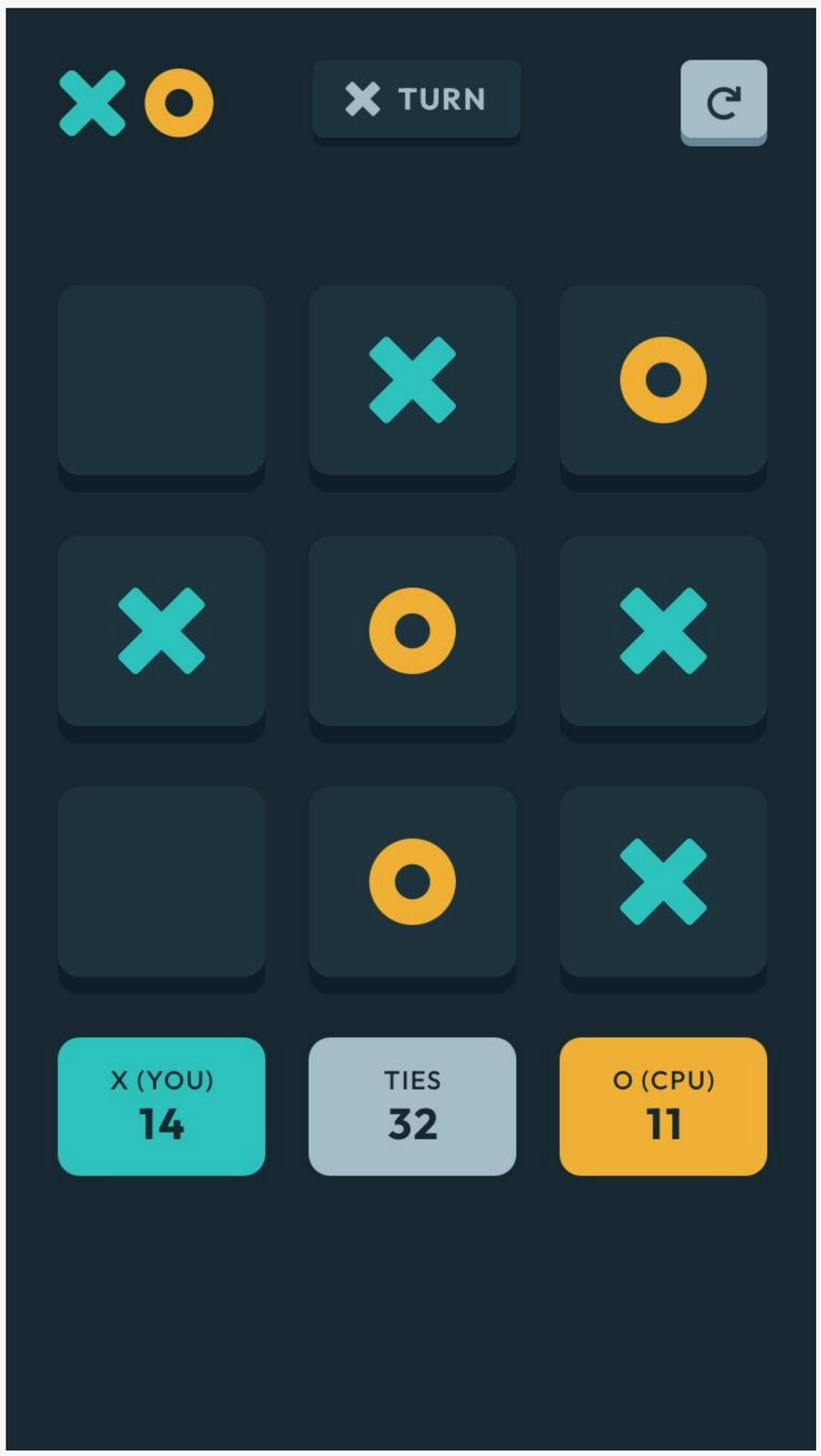


Question 2 Build Tic-Tac-Toe →

- Create a Tic-Tac-Toe game (also known as Xs and Os) using HTML, CSS, and JavaScript. The goal is to let two players take turns marking spaces in a 3×3 grid. The game should detect wins, draws, and allow players to reset and play again.

- Create a 3×3 clickable grid where two players take turns placing "X" and "O". After each move, check for a win or draw and display a message. Include a "Restart Game" button to reset the board.

Reference :



MERN Core Technologies —

01 React.Js (Frontend Library)

React.js is a JavaScript library for building interactive user interfaces. Developed by Facebook, it follows a component-based architecture, making it efficient for building scalable and dynamic web applications.

KEY TOPICS:

1. **JSX (JavaScript XML)**
    - Combines HTML with JavaScript syntax.
    - Must return a single parent element.

2. **Components :** Components are like functions that return HTML elements.
    - Functional component and class component
    - Props: passing data to components

3. **Hooks:**
    - useState() – Manage local component state
    - useEffect() – Side effects and lifecycle methods
    - useRef() – Access DOM or persist values
    - useContext() – Global state management
    - useReducer() – Alternative to useState for complex logic
    - useMemo() / useCallback() – Performance optimizations
    - Custom Hooks – Encapsulate logic in reusable functions

4. **Routing**

5. **Forms and Events**

6. **State Management**

    - Context API
    - Redux
    - Recoil

7. **Performance Optimization**

    - useMemo useCallback
    - virtualization

8. **SSR & SSG**

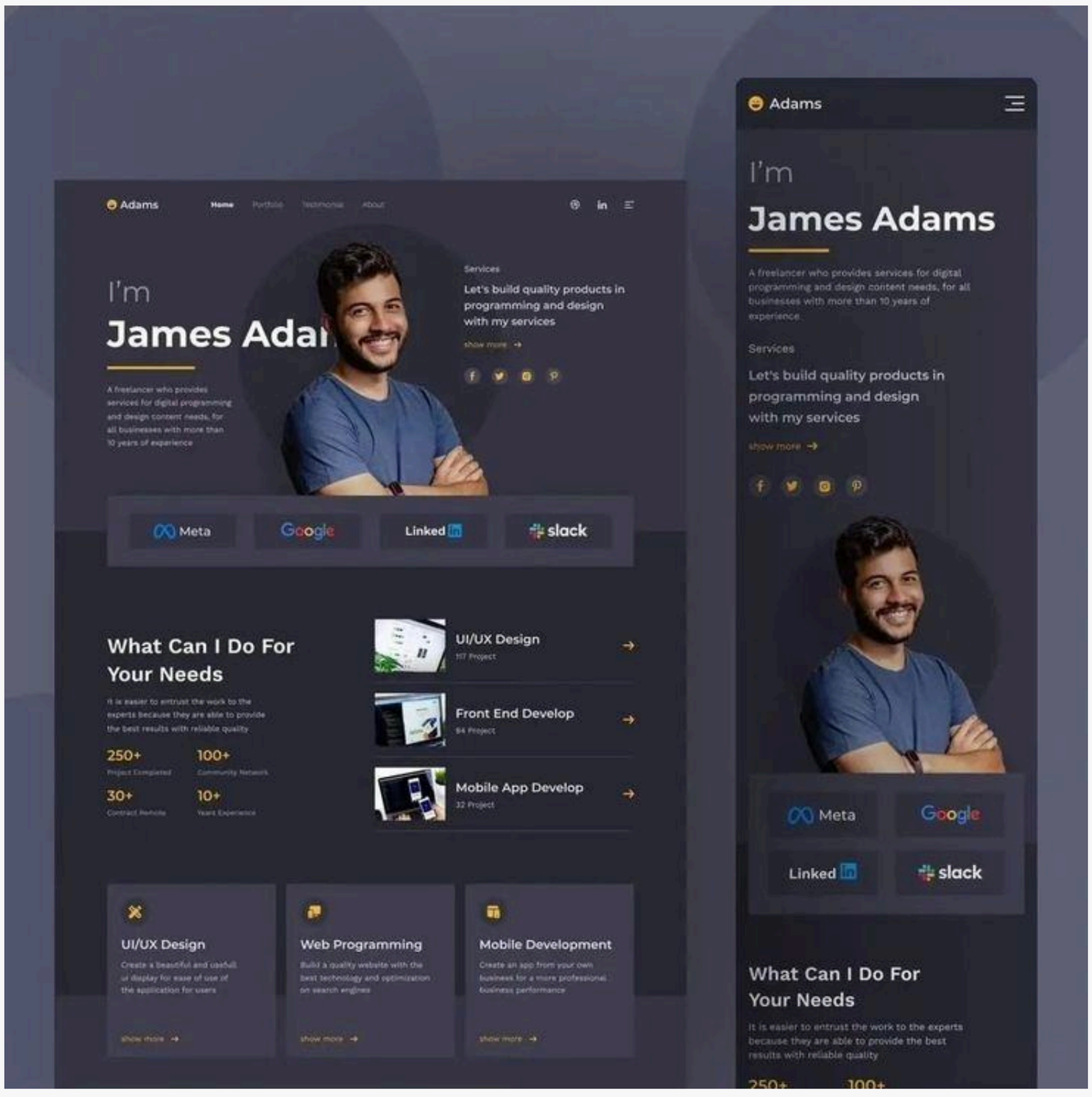
PRACTICE:

Question 1 **Your Own Portfolio Website** →

- A personal portfolio website is a single-page application (SPA) built with React to showcase your skills, projects, and contact information.Add following sections in the Portfolio -
- Header/Navigation Bar:Contains links to different sections (About, Projects, Skills, Contact).Sticky or fixed at the top for easy navigation.
- Hero Section: A brief introduction with your name, profession, and a call-to-action (e.g., "Hire Me" or "View Work").
- About Me:A short bio, education, experience, interests and a profile picture.
- Projects Section - Displays your projects with images, descriptions, and links (GitHub, Live Demo).Each project can be a card with hover effects.

- Skills/Technologies - Lists programming languages, frameworks, and tools (icons or progress bars).
- Contact Form - A form with fields for name, email, and message (connected to an API or email service).
- Footer - Social media links, copyright info, and a back-to-top button.

Reference :



02 API Concepts

APIs (Application Programming Interfaces) are essential for connecting frontend apps (like React) with backend services (like Node.js), databases, or third-party services (like payment APIs).

REST APIs and WebSockets are two widely used protocols in modern web development.

KEY TOPICS:

1. **RESTful APIs and principles:** REST (Representational State Transfer) is an architectural style for designing APIs using standard HTTP methods.
    - Stateless: Each request contains all the data needed (no memory of previous requests).
    - Resource-based: Resources are accessed via unique URLs (e.g., /api/users/1).
    - JSON is the most common data format for requests and responses.

2. **HTTP Methods:**

Method	Description	Example
GET	Read data	`GET /api/products`
POST	Create a new resource	`POST /api/products`

Method	Description	Example
PUT	Update an existing resource	`PUT /api/products/5`
DELETE	Remove a resource	`DELETE /api/products/5`

3. HTTP status code

- 20X : Success
- 30X : Redirection
- 40X : Client Errors
- 50X : Server Errors

4. Request & Response Breakdown

- Headers
- Body
- Authentication
- Query Parameters
- Route Parameters

5. API Testing Tools

- Postman: GUI tool to send API requests, test endpoints, inspect responses.
- Thunder Client: Lightweight REST client built into VSCode.
- curl: Command-line tool for making HTTP requests.

6. **Websockets :** WebSockets allow two-way real-time communication between client and server over a single long-lived TCP connection.

PRACTICE:

Question 1 **Weather App Using OpenWeatherMap API →**

- Create a frontend web application (using HTML, CSS, JavaScript — UI is not very important) that performs the following:
- user Input - Allow users to enter a city name.
- Api call - Use the GET method to call the OpenWeatherMap API and fetch weather data for the specified city.
- Display Data - Show the following weather details : City name, Temperature, Weather condition, Humidity, Wind speed

03 Database – MongoDB

MongoDB is a document-oriented NoSQL database designed for speed, scalability, and flexibility.

KEY TOPICS:

1. SQL and NoSQL Databases:

- SQL (Relational Databases) :
  - i. Structured schema (tables, rows, columns)
  - ii. Fixed data model (requires predefined schema)
  - iii. Examples: MySQL, PostgreSQL, SQL Server
- NoSQL (Non-relational Databases) :
  - i. Schema-less (flexible document structure)
  - ii. Horizontally scalable (handles large volumes of data efficiently)
  - iii. Examples: MongoDB, Cassandra, Redis

2. Introduction to MongoDB

- A document-based NoSQL database.
- Stores data in BSON (Binary JSON) format.
- Highly scalable with built-in sharding and replication.

3. Core Concepts

- Document Structure
    i. A document is a key-value pair (similar to JSON)
    ii.
```
{
    "_id": "123",
    "name": "John Doe",
    "age": 30,
    "address": { "city": "New York", "country": "USA" }
}
```
- Collections & Databases
    i. Collection : Group of documents (similar to a table in SQL)
    ii. Database : Contains multiple collections

4. CRUD Operations

5. Querying Data :

- Basic Queries: find(), findOne().
- Comparison Operators: $eq, $ne, $gt, $lt, $in.
- Logical Operators: $and, $or, $not.
- Array Operations: $push, $pull, $elemMatch.

6. Indexing & Performance

- Single Field Index: db.users.createIndex({ name: 1 })
- Compound Index: db.users.createIndex({ name: 1, age: -1 })
- Text Index (for search): db.articles.createIndex({ content: "text" })

7. Aggregation Framework

- Pipeline stages: $match, $group, $sort, $project.

PRACTICE:

Question 1 **Product Inventory Management Database Design →**

Design a MongoDB database to manage an inventory of products. Your system should support the following:

- Products Collection Each product should include:

  ```
  productId (string or number)
  name (string)
  category (string)
  price (number)
  quantityInStock (number)
  supplier (embedded document or reference)
  createdAt (date)
  ```

- Suppliers Collection Each supplier should include:

  ```
  supplierId (string or number)
  name (string)
  contact (phone or email)
  address (string)
  ```

- Use MongoDB queries (in Mongo Shell, Compass, or Mongoose) to perform the following:

  i. Insert sample products and suppliers.
  ii. Read: List all products.
  iii. Find products by category or supplier.
  iv. Find products with low stock (e.g., quantityInStock < 10).
  v. Update product details (e.g., price or stock quantity).
  vi. Delete a product by productId.

04 Node.Js (JavaScript Runtime)

Node.js is a JavaScript runtime built on Chrome's V8 engine that allows developers to execute JavaScript on the server side. It enables building scalable and high-performance backend applications.

KEY TOPICS:

1. Introduction to Node.js

- Event-driven, non-blocking I/O model
- npm (Node Package Manager) for managing dependencies.
- CommonJS modules (require & module.exports).

2. Core Modules

- fs (File System) – Read/write files.
- http – Create HTTP servers.
- path – Handle file paths.
- os – Interact with the operating system.

3. Streams & Buffers

- Readable, Writable, Duplex, and Transform streams.
- Handling large files efficiently.

4. Environment variables

- Using dotenv for configuration.

PRACTICE:

Question 1 E-Commerce Product Listing →

Design an e-commerce product listing page with shopping cart functionality. It allows users to browse products, filter/sort them, and add items to their cart. With the following key features :

- Product Display : Each product is displayed in a visually appealing card format containing: Product image, Title, Price, Short description, "Add to Cart" button.
- Filtering and Sorting :

    i. Users can filter products by category (e.g., Electronics, Clothing, Books).

    ii. Price Sorting: Options to sort products by: Price: Low to High, Price: High to Low. Search Functionality: Text search to find specific products.

- Shopping Cart :

    i. Add/Remove Items: Users can add products to cart or remove them

    ii. Quantity Adjustment: Change quantities directly in the cart

    iii. Cart Summary: Displays:Total number of items, Subtotal, Checkout button

    iv. Persistent Cart: Cart data persists using localStorage

Reference :


Nest
Browse All Categories
Hot Deals
Home
About
Shop
Vendors
Mega Menu
Blog
Pages
Contact
Fresh Vegetables
Big discount
Save up to 50% off on your first order
Subscribe
Featured Categories
Cake & Milk
Coffee & Teas
Pet Foods
Vegetables
Cake & Milk
Organic Kiwi
Peach
Red Apple
Snacks
Vegetables
Strawberry
Black plum
Custard apple
Coffe & Tea

05 Express.Js (Backend Framework)

Express.js is a fast, minimal, and flexible web application framework built on top of Node.js. It simplifies the process of creating server-side logic, handling routes, and managing middleware. It is the most popular framework for building RESTful APIs in Node.js.

- It is a lightweight Node.js framework used to build backend applications and APIs.
- It offers easy routing, middleware support, error handling, and integration with databases.

KEY TOPICS:

1. Routing

- Handling HTTP methods (GET, POST, PUT, DELETE)
- Route parameters (/users/:id)
- Query parameters (/search?q=express)

2. Middleware

- Body parsing middleware (express.json(), express.urlencoded())
- Static file middleware (express.static())
- Session middleware (express-session)
- Third-party middleware
    - i. CROS - Enables cros-origin requests
    - ii. Multer - Handles file uploads
    - iii. Express-session - Manages user sessions
    - iv. Express-rate-limit - Prevents rate-based abuse

3. Handling Request and Response

- Access request data: req.params, req.query, req.body
- Send responses: req.send(), req.json(), req.status()

4. Error Handling

- app.use((err,req,res,next) ⇒ console.error(err.stack););

PRACTICE:

Question 1 **Blog API Using Dynamic Route →**

Create a backend application using Express.js that performs the following:

- Blog Post Structure: Each blog post should contain

  id (auto-generated or UUID)
  title (string)
  author (string)
  content (string)
  createdAt (date)

- Create the following API endpoints using dynamic routing:

  GET */api/posts* - Return a list of all blog posts.
  GET */api/posts/:id* - Return a single blog post by its id.
  POST */api/posts* - Create a new blog post (pass data in request body).
  PUT */api/posts/:id* - Update a blog post by id.
  DELETE */api/posts/:id* - Delete a blog post by id.

- Add some dummy data in the database (MongoDB) and connect the API with it

06 Tools & Deployment

Git & GitHub

Git is a version control system that helps track changes in code over time. GitHub is a cloud-based hosting platform for Git repositories, enabling collaboration, version management, and open-source contribution.

KEY TOPICS:

1. Version Control System

2. Middleware

- git init - initializes a new Git repository
- git clone <url> - copies a remote repository to your local machine
- git add <file> - stages changes for commit
- git commit -m "message" - saves changes with a message
- git status - shows modified/staged files
- git log - displays commit history

3. Branching & Merging

- git branch <branch-name> - creates a new branch
- git checkout <branch-name> - switches to the branch
- git merge <branch-name> - combines changes from one branch into another

4. Remote Repositories

- git remote add origin <url> - links local repo to a remote GitHub repo
- git push -u origin main - uploads local commits to GitHub
- git pull - fetches remote changes and merges them
- git fetch- downloads remote changes without merging

5. Github Features

- Repositories: Stores project files and version history.
- Forks: Creates a personal copy of someone else's repo.
- Pull Requests (PRs): Requests to merge changes into another branch/repo.
- Issues: Tracks bugs, enhancements, and tasks.
- GitHub Actions: Automates workflows (CI/CD).
- GitHub Pages: Hosts static websites directly from a repo.

PRACTICE:

Question 1 **Push Your Portfolio Or Blog Project To GitHub** →

- Choose one of your existing projects (Portfolio Website or Blog API).
- Initialize a Git repository in your project folder.
- Create a .gitignore file and add entries as needed (e.g., node_modules, .env).
- Create a repository on GitHub.
- Connect your local project to the GitHub repository using the remote URL.
- README.md with: Project title and description Instructions to run the project locally Technologies used.

- Add, commit, and push your project files to GitHub.
- Optional: Use GitHub Pages (if it's a frontend portfolio) to publish your site.

Deployment

Deployment is the process of making your application live and accessible over the internet.

KEY TOPICS:

1. **Frontend deployment**
    - Hosting static frontend assets (HTML, CSS, JS — like from React/Vite).
    - Platforms like Vercel, Netlify, GitHub Pages, etc.

2. **Backend deployment**
    - Hosting your server-side logic (Express.js, Django, etc.).
    - Use services like Render, Railway, Fly.io, Heroku.

3. **Database deployment**
    - Hosting your database so it can be accessed by your backend online.
    - Use MongoDB Atlas for cloud-based MongoDB.

4. **Environment Variables**
    - Never hard-code sensitive info like DB URIs, API keys, or secrets.
    - Create a .env file locally for keys, secrets and credentials
    - Set these variables in the dashboard under "Environment" settings.

PRACTICE:

Question 1 — Full-Stack App Deployment →

- Deploy backend on Render, frontend on Vercel, and use MongoDB Atlas for the database with the following setup:
- Create or use an existing backend (e.g., Blog API,E-commerce).
- Connect it to MongoDB Atlas using Mongoose.
- Deploy the backend on Render and add proper environment variables on Render for: `MONGODB_URI, PORT.`
- Create or use an existing frontend project and make API calls to your deployed backend (e.g., fetch blog posts or products).
- Deploy the frontend on Vercel and make sure the frontend is calling the correct Render backend URL.
- Ensure the deployed frontend can communicate with the deployed backend and the backend can successfully read/write data to MongoDB Atlas.

1. Task Management App (Advanced To-Do List)

A full-stack task management application where users can:

- Create, edit, and delete tasks.
- Organize tasks into categories (Work, Personal, Study).
- Mark tasks as complete/incomplete.
- Filter tasks by status (All, Completed, Pending).
- User authentication (Signup, Login, Logout).

HOW TO BUILD IT?

1. Frontend (React.js)

- Create components: TaskList, TaskItem, AddTask, AuthForm.
- Use useState, useEffect, and useContext for state management.
- Implement React Router for navigation (/login, /signup, /dashboard).

2. Backend (Node.js + Express.js)

- REST API endpoints:
  i. POST `/api/tasks` (Create task)
  ii. GET `/api/tasks` (Fetch tasks)
  iii. PUT `/api/tasks/:id` (Update task)
  iv. DELETE `/api/tasks/:id` (Delete task)
- User authentication using JWT (JSON Web Tokens).

3. Database (MongoDB)

- Two collections: users and tasks
- Schema for tasks:

```
{
 title: String,
description: String,
 category: String,
 completed: Boolean,
 userId: ObjectId // Reference to user who created it
}
```

4. Deployment

- Frontend: Host on Vercel/Netlify.
- Backend: Deploy on Render/Railway.
- Database: Use MongoDB Atlas.

---

2. E-Commerce Store (Basic Amazon Clone)

A simple e-commerce platform with:

- Product listing with images, prices, and descriptions.
- Shopping cart functionality (add/remove items).
- User authentication (Signup, Login).
- Checkout page (simulated payment).

HOW TO BUILD IT?

1. Frontend (React.js)

- Components: ProductList, ProductCard, Cart, Checkout.
- Use Redux Toolkit for cart state management.
- Fetch products from backend API (GET /api/products).

2. Backend (Node.js + Express.js)

- API Endpoints:
  - i. GET `/api/products` (List all products)
  - ii. POST `/api/cart` (Add to cart)
  - iii. GET `/api/cart` (View cart items)
- Use Stripe API (or mock payment).

3. Database (MongoDB)

- Collections: products, users, orders.
- Schema for products:

```
{
  name: String,
  price: Number,
  description: String,
  image: String,
  category: String
}
```

4. Deployment

- Frontend: Vercel.
- Backend: Render.
- Database: MongoDB Atlas.

---

3. Social Media Dashboard (Mini Twitter Clone)

A lightweight social media app where users can:

- Post short messages (tweets).
- Like/comment on posts.
- Follow/unfollow users.
- View a feed of posts from followed users.

HOW TO BUILD IT?

1. Frontend deployment

- Components: Feed, Post, Sidebar, UserProfile.
- Use React Query for API data fetching.
- Real-time updates with Socket.io (optional).

2. Backend (Express.js)

- REST API:
  - i. POST `/api/posts` (Create post)
  - ii. GET `/api/posts` (Fetch posts)
  - iii. POST `/api/likes` (Like a post)
  - iv. POST `/api/follow` (Follow a user)

- Authentication via JWT.

3. Database (MongoDB)

- Collections: users, posts, likes, follows.
- Schema for posts:

```
{
    content: String,
    userId: ObjectId,
    likes: Number,
    comments: [{ text: String, userId: ObjectId }]
}
```

4. Deployment

- Frontend: Netlify.
- Backend: Railway.
- Database: MongoDB Atlas.

BOSSCODER ACADEMY

WHY BOSSCODER?

01 STRUCTURED INDUSTRY-VETTED CURRICULUM

Our curriculum covers everything you need to get become a skilled software engineer & get placed.

02 1:1 MENTORSHIP SESSIONS

You are assigned a personal mentor currently working in Top product based companies.

03 2200+ ALUMNI PLACEMENT

2200+ Alumni placed at Top Product-based companies.

04 24 LPA AVERAGE PACKAGE

Our Average Placement Package is **24 LPA** and highest is **98 LPA**

Niranjan Bagade
Software Engineer,
British Petroleum

10 Years
Experience

NICE Software Eng. Specialist	Hike **83%**	**British Petroleum** Software Engineer

Dheeraj Barik
Software Engineer 2,
Amazon

2 Years
Experience

Infosys Software Engineer	Hike **550%**	**Amazon** SDE 2

EXPLORE MORE